RESTORATION PROTOTYPES — SHARED RESEARCH & PLAYTEST PROTOCOL

v1.0 | Four comparable Roblox experiments

Status: Proposed; not implemented, measured, or approved.
Date: 2026-11-06
Applicability: Prototypes A, B, C, D only. This document does not authorize new features.

1. Research objective and falsifiable hypothesis

Question: Which of four restoration interactions delivers the strongest observable satisfaction and voluntary replay relative to actual engineering and content-authoring cost?

Hypothesis: At least one interaction will be understandable without instruction, enable most new testers to complete a short task, and motivate a substantial share to replay **without currencies or upgrades**.

Disconfirmation: Testers repeatedly need explanation, do not complete, report the action is tedious, or only express interest after requesting external rewards. A visually attractive final screenshot is not sufficient validation.

2. Control the experiment

Variable	Required policy
Platform	Roblox Studio test place; each prototype isolated from Thaw a Creature
Audience	First-time testers unfamiliar with the particular prototype
Sample	Suggested 10–15 participants per prototype for qualitative issue discovery; **not** statistical proof
First round	Target 45–90 sec; tune content density; measure actual time
Loop	Observe mess → interact → visible change → 100% transformation → replay choice
Controls	Desktop and touch screen usable; document device for every test
Team	Same developer skill assumptions and time-logging conventions for each
Progression	No currency, no persistent inventory, no upgrades, no automation
Social	Single player P0; no networking benefits or social rewards in one prototype only
Visual fidelity	Similar low-poly/blockout clarity and equivalent basic sound feedback
Guidance	One short in-world instruction, no forced tutorial cutscene
Second round	Same action, reseeded or changed content; no new mechanic introduced

Test order can bias results. If using the same participants, rotate which prototype they try first. Do not present A then C to everyone. An additional within-prototype comparison for C (hidden vs disclosed result) is optional and **must be logged as a separate variant**, because C intentionally introduces anticipation not present in A.

3. Common state & server authority contract

All prototypes use the logical states:

`READY → ACTIVE → COMPLETE → REPLAY_OFFERED → RESETTING → ACTIVE`

- Each round has an immutable, server-generated `RoundId`, `StartedAt`, configuration version, and completion condition.
- The **server alone** accepts cleaning, placement, or reveal progress and decides completion. Clients may predict cosmetic feedback but must reconcile to authoritative results.
- The server validates action rate, legitimate target ID, distance, round ID, active state, and the target's not-already-completed state. Bad or repeated requests cannot increment progress twice.
- Reset creates a new round ID and resets all relevant objects, not merely the HUD percentage. Requests from old rounds are rejected.
- Do not use DataStore for P0. Test-session counters are in memory and can be printed as structured logs at the end.
- When a player leaves or their character resets mid-task, clean up held objects, active actions, and session state. The prototype should be recoverable with a fresh round.

Suggested small architecture (not mandatory exact filenames)

```
ReplicatedStorage/
  RestorationShared/Config.lua
  RestorationShared/Remotes/ActionRequested
  RestorationShared/Remotes/StateChanged
ServerScriptService/
  RestorationServer.server.lua        -- bootstrap only
  RestorationModules/RoundService.lua
  RestorationModules/ActionValidator.lua
  RestorationModules/MetricLogger.lua
StarterPlayer/StarterPlayerScripts/
  RestorationClient.client.lua        -- bootstrap only
  RestorationClientModules/InputController.lua
  RestorationClientModules/FeedbackController.lua
  RestorationClientModules/HUDController.lua
Workspace/
  PrototypeArena/
```

Prototype-specific modules should replace/extend the action module instead of creating redundant script clones attached to every prop. This shared architecture is a guide, not a dependency on an elaborate framework.

4. Shared telemetry event definitions

Event	Required fields	When emitted
`RoundStarted`	SessionId, Prototype, Variant, RoundId, Device, ConfigVersion, timestamp	Server initializes playable round
`FirstValidAction`	RoundId, elapsedSeconds	First server-accepted meaningful interaction
`ProgressChanged`	RoundId, progress01, elapsedSeconds	Server accepts progress; sample or coalesce; avoid console spam
`RoundCompleted`	RoundId, elapsedSeconds, acceptedActions	Server marks 100% complete
`ReplayOffered`	RoundId, elapsedSeconds	Clear repeat choice is displayed
`ReplayChosen`	previous RoundId, new RoundId	Player voluntarily starts second round
`SessionEnded`	RoundId, result, elapsedSeconds	Player leaves, tester stops, or recording ends
`RequestRejected`	reason, actionType	Aggregate counts, not high-frequency per event logs

Use a session-scoped random test ID, not personal identifiers in shared reports. No player-age inference, purchase data, or secret identifiers. Record tester feedback on a separate consented sheet; do not add network tracking without approval.

5. Common performance and accessibility checks

- Desktop: mouse + keyboard; mobile: touch controls in a real or emulated mobile viewport. Gamepad is a desirable check if time allows, not P0 in this pack.
- Action targets must be visually legible. Do not rely exclusively on red vs green. Use text/progress numbers and shape/visibility change.
- Avoid flashing effects; allow basic VFX/sound toggle if any test participant needs it.
- Art budget first pass: aim for an understandable blockout with stable performance on a mid-range phone. Collect FPS and frame time; do not claim a platform-wide performance guarantee from one emulator.
- No client should be able to increase total progress by spamming remotes, forging completed status, or acting beyond reach.

6. Measurement formulas and interpretation

Metric	Exact definition	Proposed first-pass interpretation
Time to first action	`FirstValidAction - RoundStarted`	Desirable median <=10 sec
Completion rate	Unique first rounds completed / eligible first rounds started	Exploratory target >=75%
Completion duration	`RoundCompleted - RoundStarted` on completed rounds	Aim 45–90 sec, but do not optimize solely for duration
Voluntary replay rate	Players who choose an offered second round / players shown a replay offer	Exploratory target >=50%
Self-reported satisfaction	Standard question, 1–5: “How satisfying was the transformation itself?”	Explore average >=4.0

Metric	Exact definition	Proposed first-pass interpretation
Comprehension failure	Tester needs explanation or uses wrong action >2 times	Flag repeated patterns, not just mean
Content authoring effort	Minutes to construct and QA the second playable variation	Lower is better at comparable quality
Actual implementation time	Focused work hours P0 + bugs + mobile pass	Count debugging and failed experiments

Warning: These cutoffs are *hypotheses chosen for decision-making*, not Roblox market benchmarks. A 10–15-person convenience sample cannot establish true retention, product-market fit, revenue, or significance. Include raw numbers (e.g., 8/12), not percentages alone. Ask players to show behavior rather than relying only on stated intent.

7. Common scripted observation protocol

1. Ask the tester to start without prior game explanation beyond the on-screen label.
2. Record the first meaningful action and whether it occurred without prompting.
3. Observe areas of confusion, repeated actions, device issues, navigation problems, and unintended successes.
4. Record the completion result and elapsed time.
5. Offer a neutral **“Try another round”** button beside a neutral **“Finish”** button. Do not advertise prizes or create false urgency.
6. Observe whether a second round begins voluntarily; do not push.
7. Ask the same two questions: “How satisfying was the transformation itself, from 1 to 5?” and “What part felt best or worst?”
8. Record the real time needed to author a second content variation of comparable quality.

8. Cross-prototype decision scorecard

Complete only after the same kind of playtest exists for each. A suggested rubric, **not a prediction**, is:

Criterion	Weight	Scoring principle
Voluntary replay	30%	Higher observed replay counts, accounting for sample size
First-round comprehension/completion	25%	Can new players act, understand progress, and finish?
Satisfying transformation	20%	Directly observed reactions + common 1–5 question
Focused development cost	15%	Actual implementation hours and technical risks
Cost of content expansion	10%	Real second-variant authoring time and QA

Keep qualitative notes beside numbers. A one-point difference on a synthetic weighted score is not evidence that one prototype will earn more on Roblox. Any candidate that fails the **core interaction** should not be rescued by hypothetical monetization systems.

9. Decision gates and stop rules

Gate G0: Input and visual clarity. First-time tester can perform action without a verbal explanation. If not, fix cues before adding progression.

Gate G1: Functional correctness. No double-counted progress, no stale-round events, no stuck held objects, no false 100%, no forced crashes.

Gate G2: Emotional payoff. Players can clearly point to what became better and at least some voluntarily complete a second round.

Gate G3: Content scalability. A second variant must be built with measured effort. If authoring 10 variants requires 10 different systems, the product is not low-cost despite the P0 prototype.

Gate G4: Selection. Compare actual data, choose one **if evidence differentiates**, or run a targeted second experiment if results are ambiguous. Do not plan upgrades, currencies, or retention systems before this decision.

10. Sources and scope

Roblox Creator Hub (technical support, verified 2026-11-06):

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/ProximityPrompt
- https://create.roblox.com/docs/reference/engine/classes/CollectionService
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService
- https://create.roblox.com/docs/reference/engine/classes/EditableImage

Market motivation was identified in the separately prepared `Roblox_Market_Intelligence_Research_Report_2026-11-06.md`. This shared protocol does not claim that reference games launched with these minimal features, or that their development costs are publicly known.